SINCE 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

MIT-WPU

॥ विश्वशान्तिर्भुवं ध्रुवा ॥

Supported by UNESCO

UNESCO Chair for Human Rights, Democracy, Peace & Tolerance
World Peace Centre (Alandi) Pune, India

Unit 4

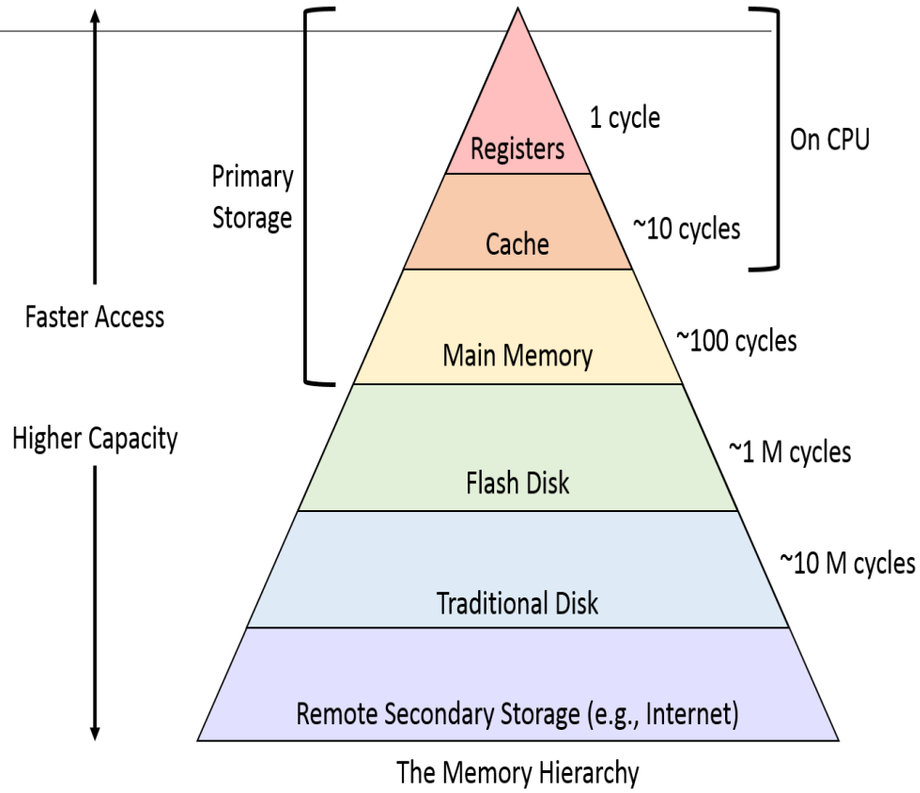
Memory Management

Prof. Vilas C. Rathod
Department of Computer Engineering
School of Polytechnic & Skill Development

Course Outcomes

Main Memory

- The main memory is central to the operation of a Modern Computer.
- Main Memory is a large array of words or bytes, ranging in size from hundreds of thousands to billions.
- Main memory is a repository of rapidly available information shared by the CPU and I/O devices.
- Main memory is the place where programs and information are kept when the processor is effectively utilizing them.
- Main memory is associated with the processor, so moving instructions and information into and out of the processor is extremely fast.
- Main memory is also known as RAM (Random Access Memory).
- This memory is volatile. RAM loses its data when a power interruption occurs.



The Memory Hierarchy

Memory management

- Memory management is the process of controlling and coordinating a computer's main memory.
- It ensures that blocks of memory space are properly managed and allocated so the operating system (OS), applications and other running processes have the memory they need to carry out their operations.
- In a multiprogramming computer, the Operating System resides in a part of memory, and the rest is used by multiple processes.
- The task of subdividing the memory among different processes is called Memory Management.
- Memory management is a method in the operating system to manage operations between main memory and disk during process execution.
- The main aim of memory management is to achieve efficient utilization of memory.

Why Memory Management is Required?

- Allocate and de-allocate memory before and after process execution.
- To keep track of used memory space by processes.
- To minimize fragmentation issues.
- To proper utilization of main memory.
- To maintain data integrity while executing of process.

Logical vs. Physical Address Space

- **Logical address** – generated by the CPU; also referred to as **virtual address**
- **Physical address** – address seen by the memory unit

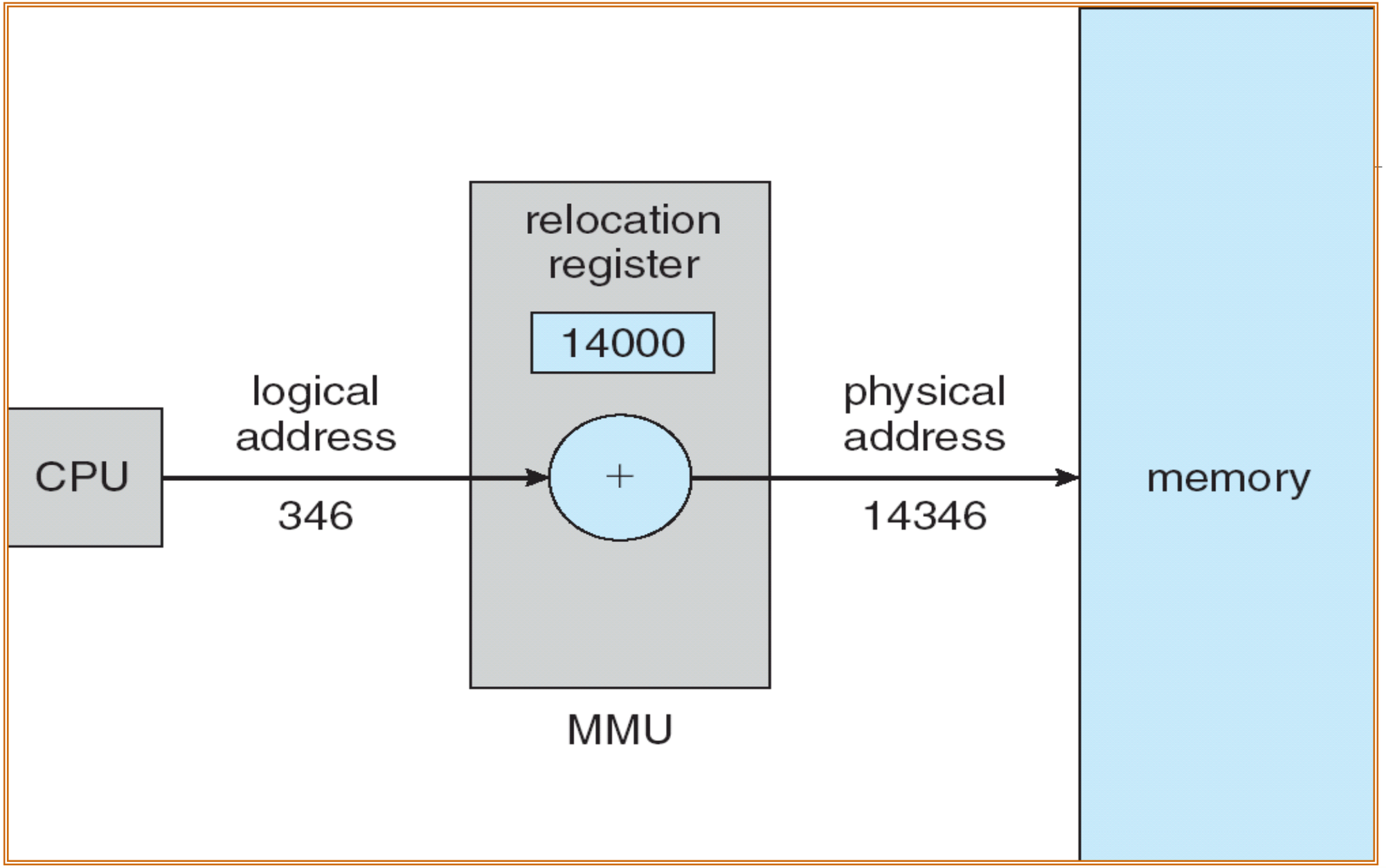
The set of all logical addresses used by a program composes the **logical address space**

The set of all corresponding physical addresses composes the **physical address space**

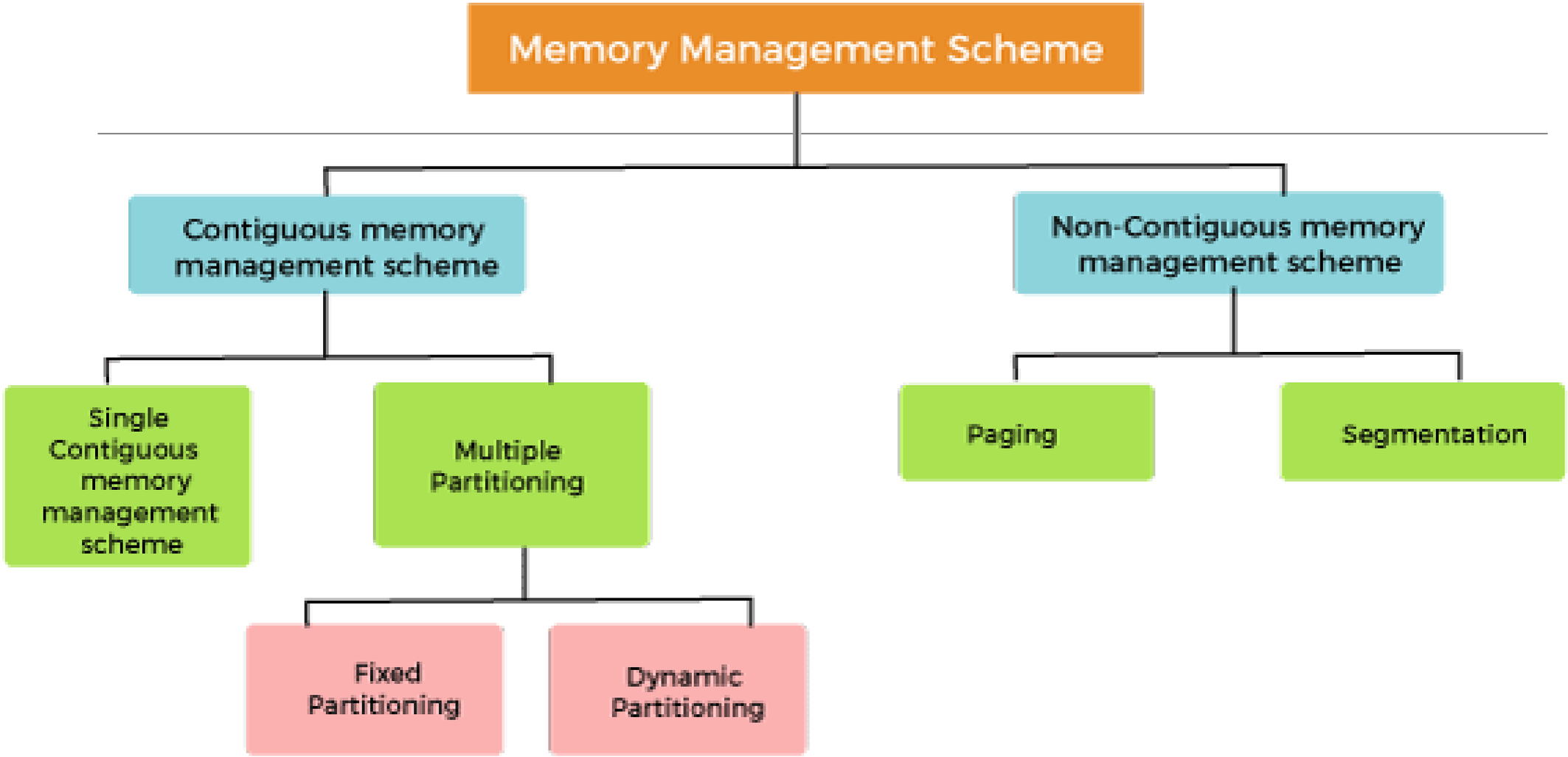
Logical vs. Physical Address Space

- The run time mapping of logical to physical addresses is handled by the memory-**management unit, MMU**.
- The base register is also known as a **relocation register**, whose value is added to every memory request at the hardware level.
- The user program deals with logical addresses, it never sees the real physical addresses

Dynamic relocation using relocation register



Memory Management Techniques:



Classification of memory management schemes

Memory Management Techniques:

Contiguous memory management schemes:

- In a Contiguous memory management scheme, each program occupies a single contiguous block of storage locations,
- i.e., a set of memory locations with consecutive addresses.

Single contiguous memory management schemes:

- In this scheme, the main memory is divided into two contiguous areas or partitions.
- The operating systems reside permanently in one partition, generally at the lower memory, and the user process is loaded into the other partition.

Memory Management Techniques:

Multiple Partitioning:

- The single Contiguous memory management scheme is inefficient as it limits computers to execute only one program at a time resulting in wastage in memory space and CPU time.
- The problem of inefficient CPU use can be overcome using multiprogramming that allows more than one program to run concurrently.
- To switch between two processes, the operating systems need to load both processes into the main memory.
- The operating system needs to divide the available main memory into multiple parts to load multiple processes into the main memory.
- Thus multiple processes can reside in the main memory simultaneously.

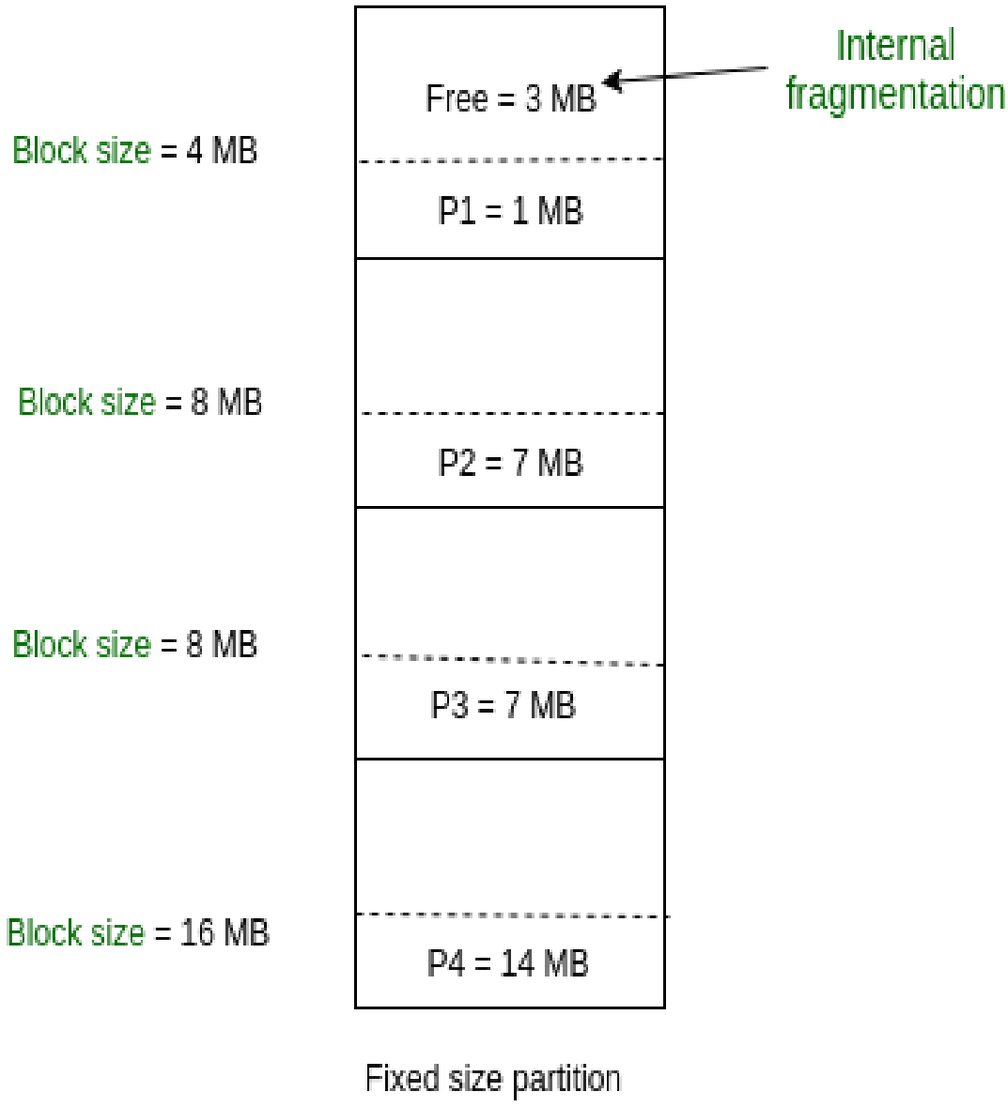
Memory Management Techniques: **Multiple Partitioning**

The multiple partitioning schemes can be of two types:

- Fixed Partitioning
- Dynamic Partitioning

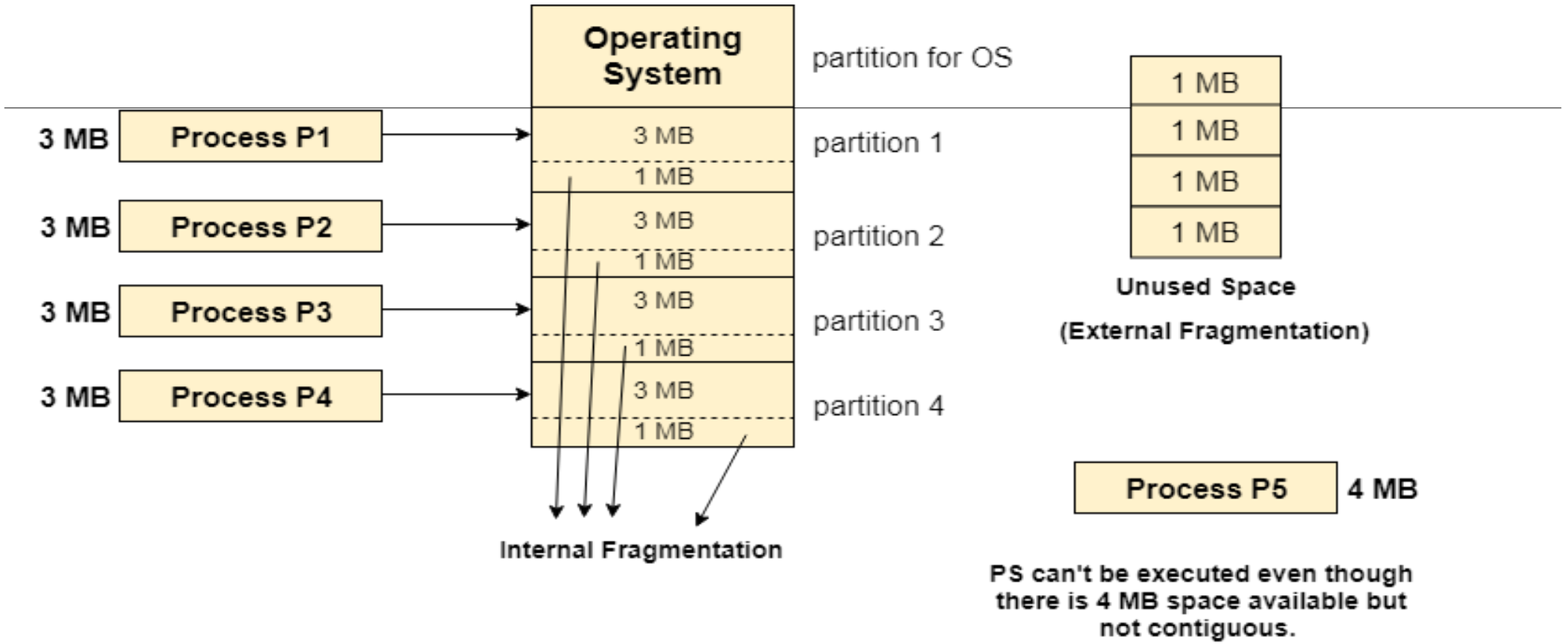
Fixed Partitioning

- This is the oldest and simplest technique used to put more than one processes in the main memory.
- One of the simplest methods for allocating memory is to divide memory into several fixed sized partitions Each partition may contain exactly one process.
- Thus, the degree of multiprogramming is bound by the number of partitions.
- In this multiple partition method when a partition is free, a process is selected from the input queue and is loaded into the free partition.
- When the process terminates, the partition becomes available for another process.
- In this partitioning, numbers of partitions (non-overlapping) in RAM are fixed but size of each partition may or may not be same.
- As it is contiguous allocation, hence no spanning is allowed. Here partition is made before execution or during system configure.



Fixed size partition

Fixed Partitioning



Fixed Partitioning

(Contiguous memory allocation)

Fixed Partitioning

Advantages of Fixed Partitioning memory management schemes:

- Simple to implement.
- Easy to manage and design.

Disadvantages of Fixed Partitioning memory management schemes:

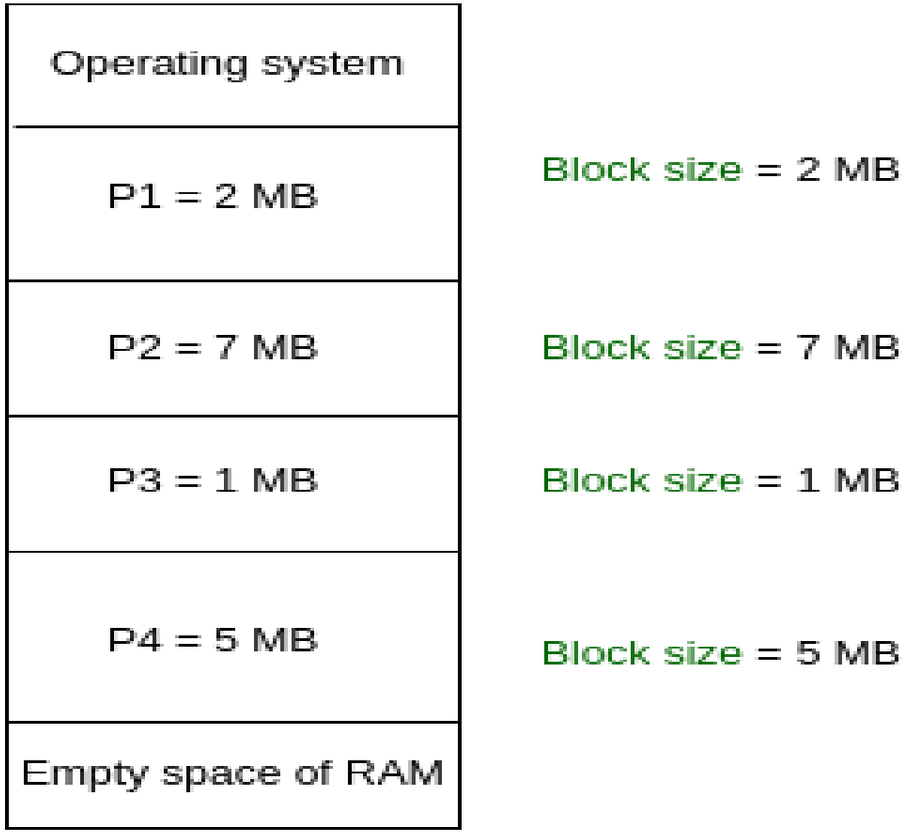
- This scheme suffers from internal fragmentation.
- The number of partitions is specified at the time of system generation.

Dynamic Partitioning

- The dynamic partitioning was designed to overcome the problems of a fixed partitioning scheme.
- In a dynamic partitioning scheme, each process occupies only as much memory as they require when loaded for processing.
- Initially, all memory is available for user processes and is considered one large block of available memory as a hole.
- In general as mentioned, the memory blocks available comprise a set of holes of various sizes scattered throughout memory.
- When a process arrives and needs memory, the system searches the set for a hole that is large enough for this process.
- If the hole is too large, it is split into two parts. One part is allocated to the arriving process; the other is returned to the set of holes. When a process terminates, it releases its block of memory, which is then placed back in the set of holes.
- If the new hole is adjacent to other holes, these adjacent holes are merged to form one larger hole.

Dynamic Partitioning

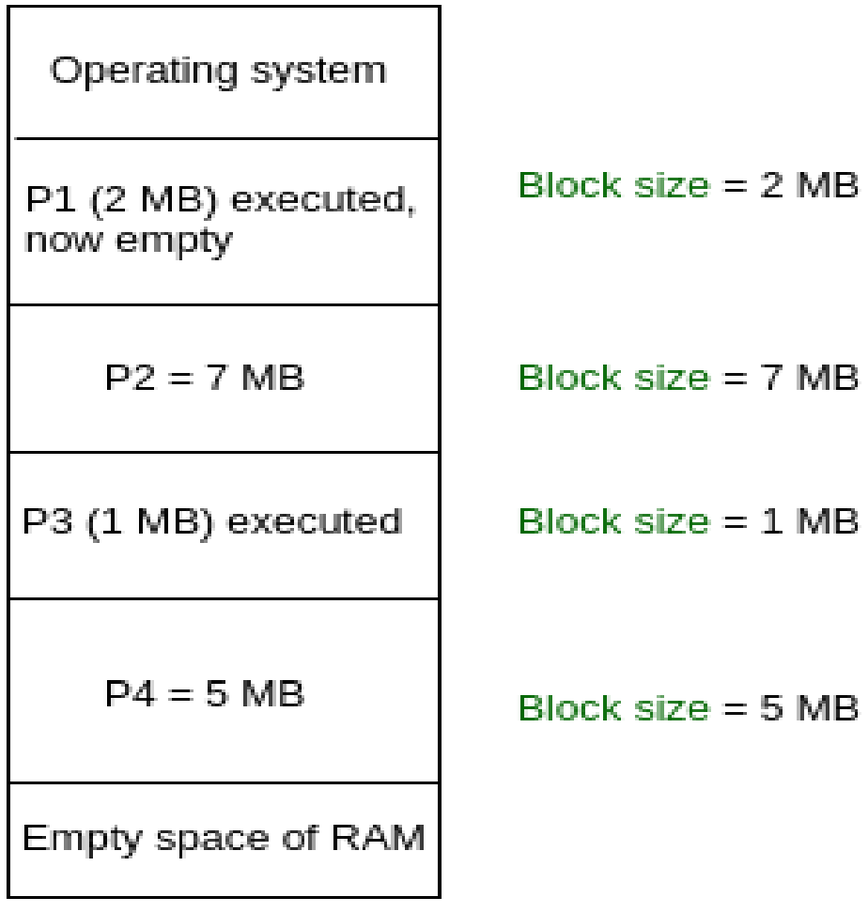
Dynamic partitioning



Partition size = process size
So, no internal Fragmentation

After allocation

Dynamic partitioning



Partition size = process size
So, no internal Fragmentation

After execution of some processes

Memory Allocation (in case of dynamic partition)

1. First-fit: Allocate the *first* hole that is big enough.

- The first-fit algorithm searches for the **first free partition** that is **large enough** to accommodate the process.
- The operating system **starts searching from the beginning** of the memory and **allocates the first free partition** that is large enough to fit the process.
- For example, suppose we have the following memory partitions:

| 10 KB | 20 KB | 15 KB | 25 KB | 30 KB |

Now, a process requests 18 KB of memory. The operating system starts searching from the beginning and finds the first free partition of 20 KB. It allocates the process to that partition and keeps the remaining 2 KB as free memory.

Memory Allocation (in case of dynamic partition)

2. Best-Fit

- The **best-fit algorithm** searches for the **smallest free partition** that is large enough to accommodate the process.
- The operating system searches the entire memory and selects the free partition that is closest in size to the process.
- For example, suppose we have the following memory partitions:

| 10 KB | 20 KB | 15 KB | 25 KB | 30 KB |

Now, a process requests 18 KB of memory. The operating system searches for the smallest free partition that is larger than 18 KB, and it finds the partition of 20 KB. It allocates the process to that partition and keeps the remaining 2 KB as free memory.

Memory Allocation (in case of dynamic partition)

3. Worst-Fit:-

- The worst-fit algorithm searches for the **largest free** partition and **allocates the process to it.**

- For example, suppose we have the following memory partitions:

| 10 KB | 20 KB | 15 KB | 25 KB | 30 KB |

Now, a process requests 18 KB of memory.

The operating system searches for the largest free partition, which is 30 KB.

It allocates the process to that partition and keeps the remaining 12 KB as free memory.

Advantages & Disadvantages of Variable Partitioning

Advantages of Variable Partitioning –

No Internal Fragmentation

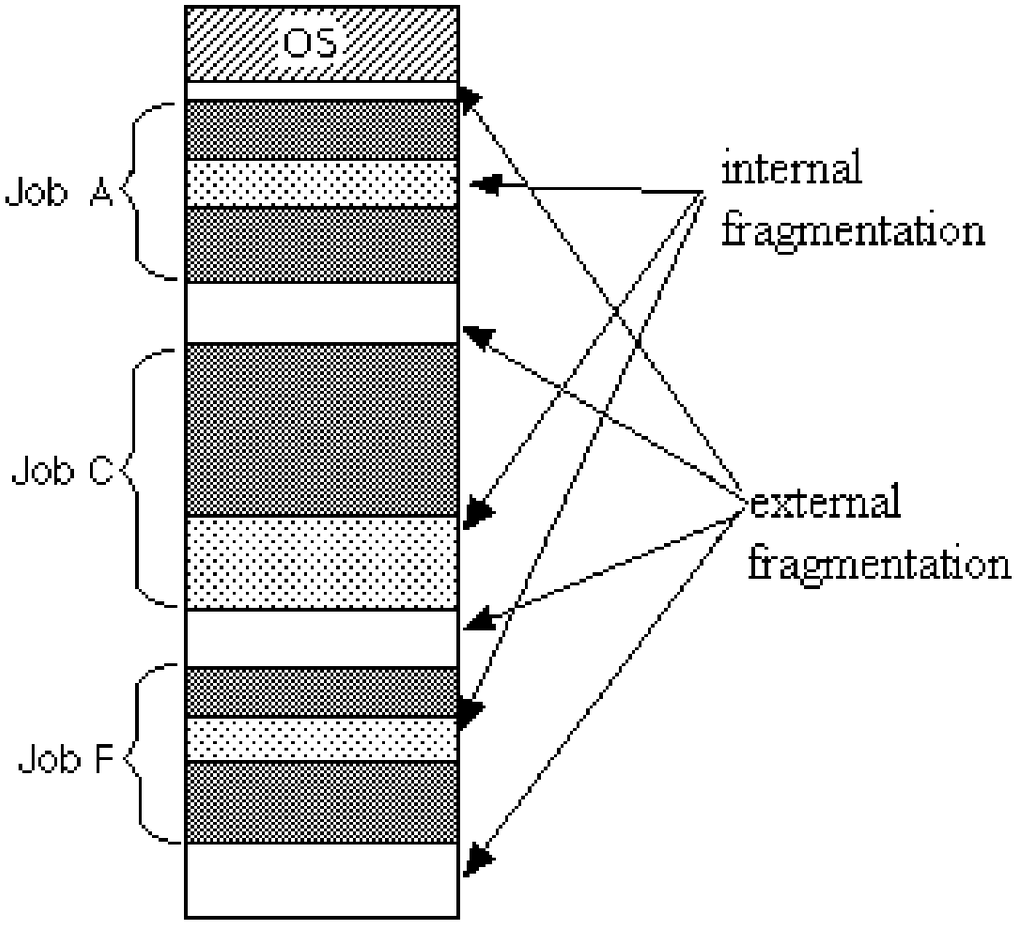
No restriction on Degree of Multiprogramming

No limitation on size of process

Disadvantages of Variable Partitioning –

Difficult Implementation

External Fragmentation: External fragmentation means that the available memory is broken up into lots of little pieces, none of which is big enough to satisfy the next memory requirement, although the sum total could.



Free Space Management Techniques

- Since disk space is limited, we need to reuse the space from deleted files for new files, if possible.
- To keep track of free disk space, the system maintains a free space list.
- The free-space list records all *free* disk blocks-those not allocated to some file or directory.
- To create a file, we search the free-space list for the required amount of space and allocate that space to the new file. This space is then removed from the free-space list. When a file is deleted, its disk space is added to the free-space list.

Free Space Management Techniques

1. Bit Map

2. Linked List

Bit Map for Dynamic Partitioning

- The main task of the operating system is to keep track of whether the **partition is free or filled**. For this purpose, the operating system also manages another data structure that is called **bitmap.**
- Frequently, the free-space list is **implemented as a bit map** or a bit vector.
- A Bitmap or Bit Vector is **series or collection of bits** where each bit corresponds to a disk block.

  Each block is represented by **1 bit.**
  - **If the block is free, the bit is 1;**
  - **if the block is allocated, the bit is 0**.
- For example, consider a disk where blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17,18, 25, 26, and 27 are free and the rest of the blocks are allocated. The free

  A **string of 0s** in the bitmap shows that there is a **hole** in the relative Allocation unit

  While, the **string of 1s** represents the **process** in the relative allocation unit.

space bit map would be:

00111100111111000110000011100000 ...

Bit Map for Dynamic Partitioning

Figure 1 (where green blocks are allocated) can be represented by a bitmap of 16 bits as: **0000111000000110**.

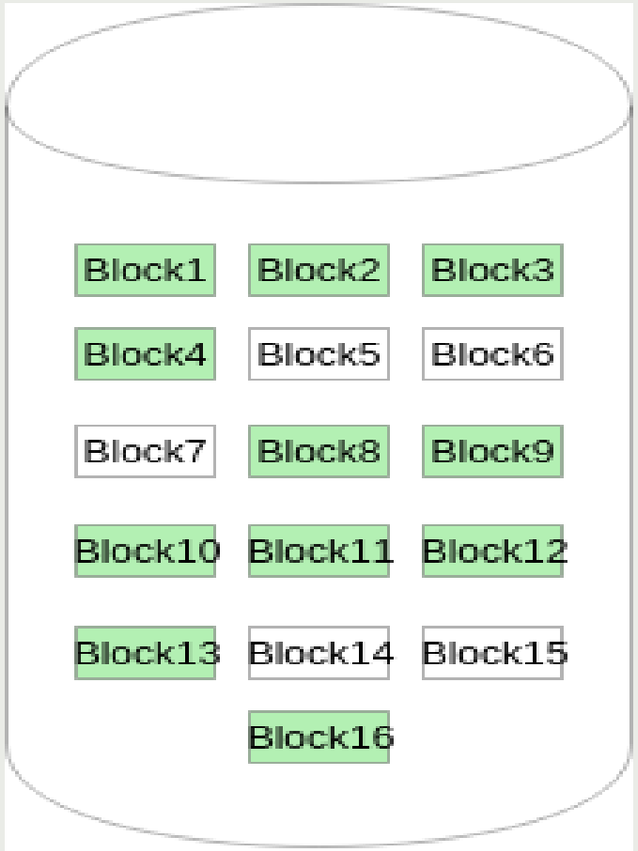


Figure - 1

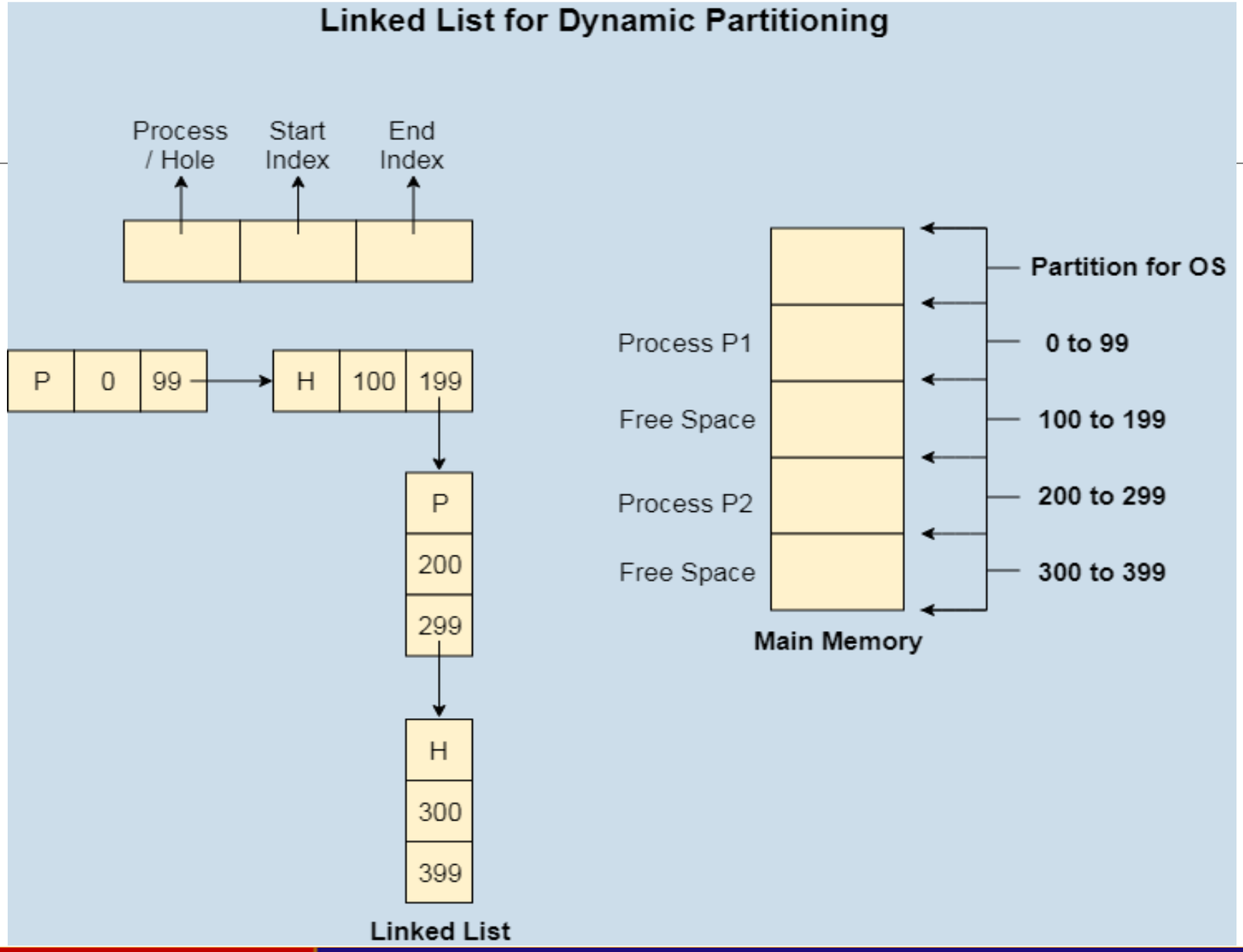
Advantages & Disadvantages of using Bitmap

Advantages:

- The main advantage of this approach is its relative simplicity and its efficiency in finding the first free block or n consecutive free blocks on the disk.

- **Disadvantages**
- Decreases the degree of multiprogramming as well as throughput.
- Takes a huge amount of time for searching which makes the system inefficient to some extent

Linked List for Dynamic Partitioning



Linked List for Dynamic Partitioning

- The better and the most popular approach to **keep track the free or filled** partitions is using **Linked List**
- Approach to free-space management is to link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory.
- This first block contains a **pointer to** the next free disk block, and so on.
- For example: blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the blocks are allocated.
- In this situation, we would keep a pointer to block 2 as the first free block. Block 2 would contain a pointer to block 3, which would point to block 4, and so on.

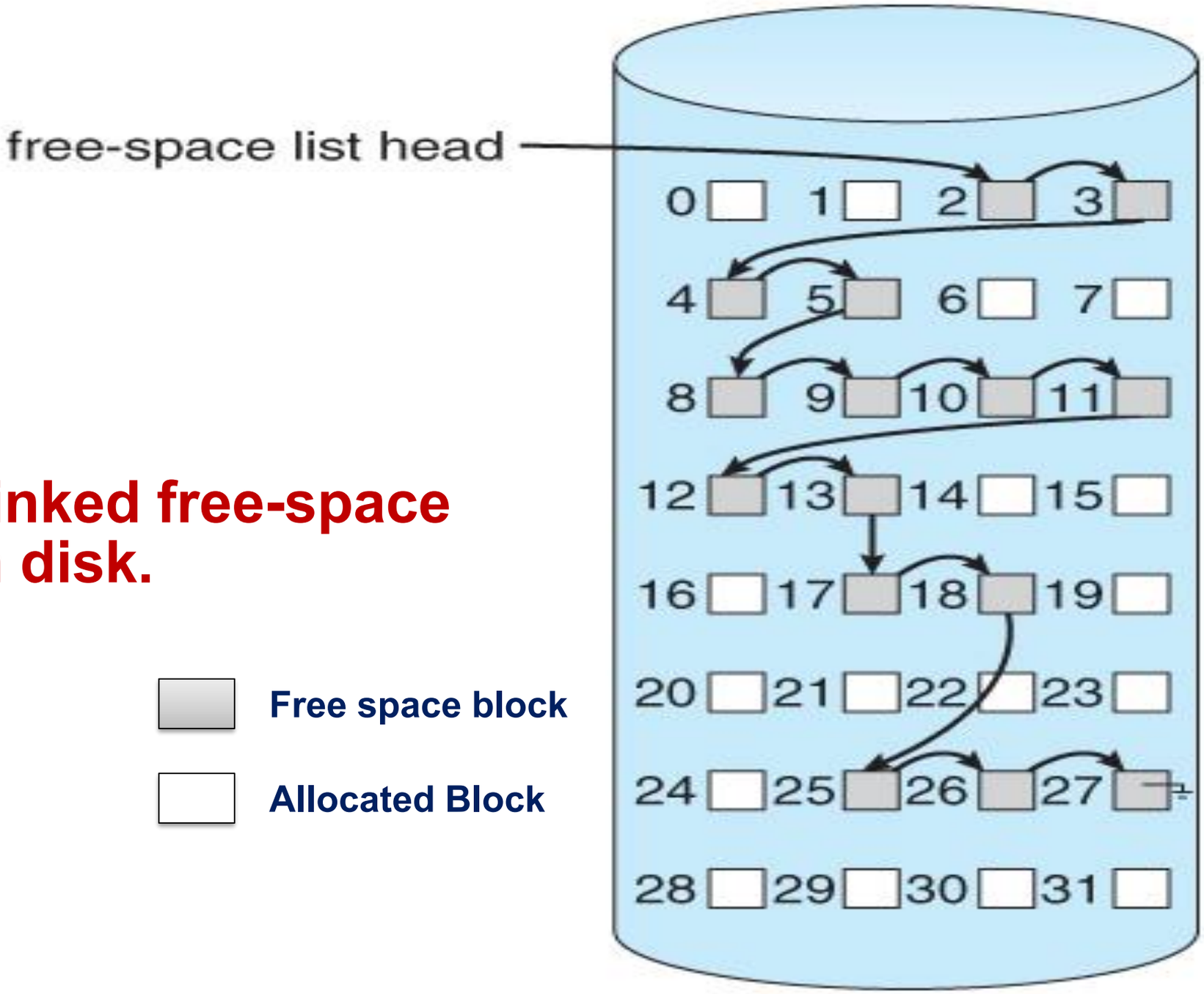
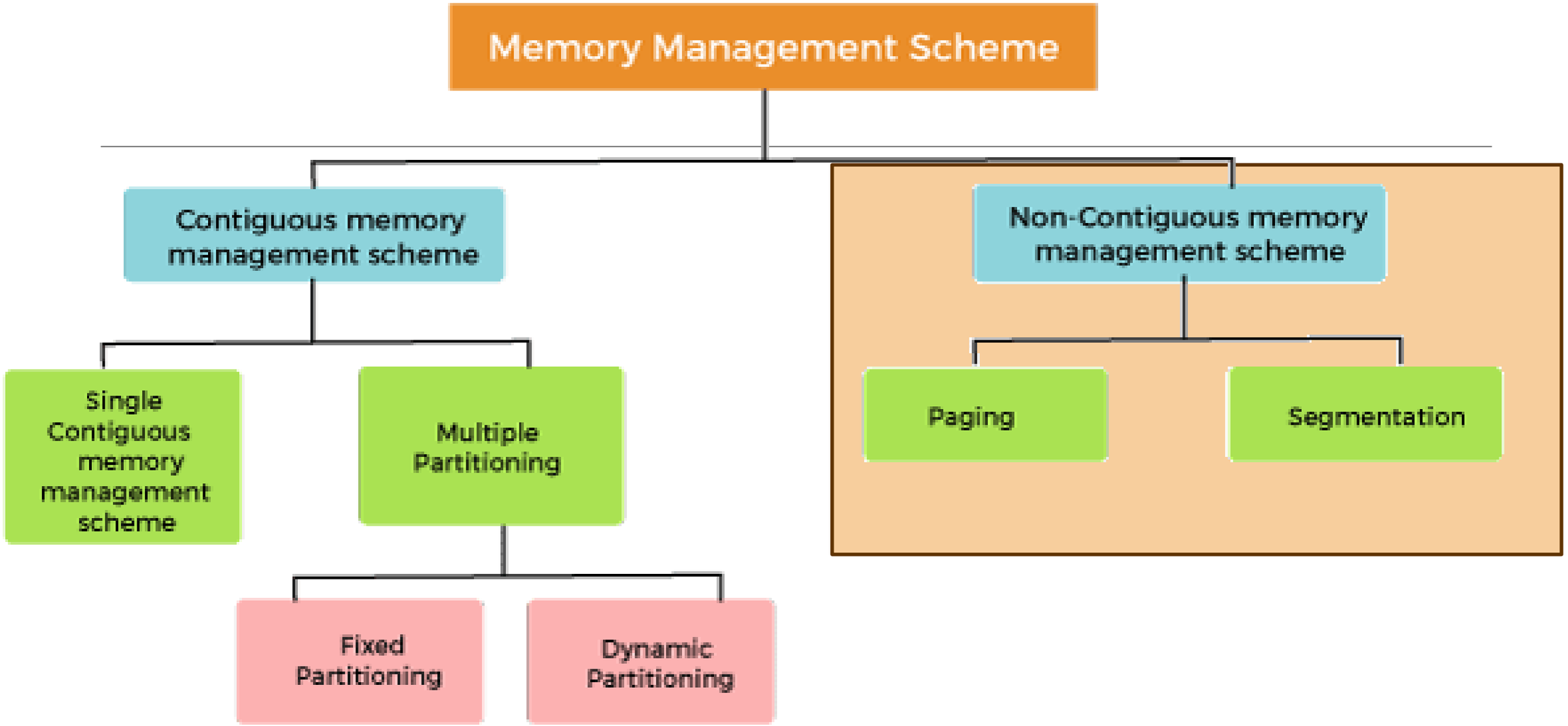


Fig. Linked free-space list on disk.

Memory Management Techniques:



Classification of memory management schemes

Non-Contiguous memory management schemes:

- In a Non-Contiguous memory management scheme, the **program is divided into different blocks** and **loaded at different portions of the memory** that need not necessarily be adjacent to one another.

- Non-contiguous memory allocation also reduces memory waste caused by internal and external fragmentation because it utilizes the memory gaps left by internal and external fragmentation.

Example:- suppose you have 30 MB process you need 30 MB space for execute purpose
but you don't have much space available for execution purpose.
So you go through non contiguous MM schemes & divide 30 MB process into 3 parts
i.e. 10 MB each & execute it.

Paging: Concept, Structure of Page table

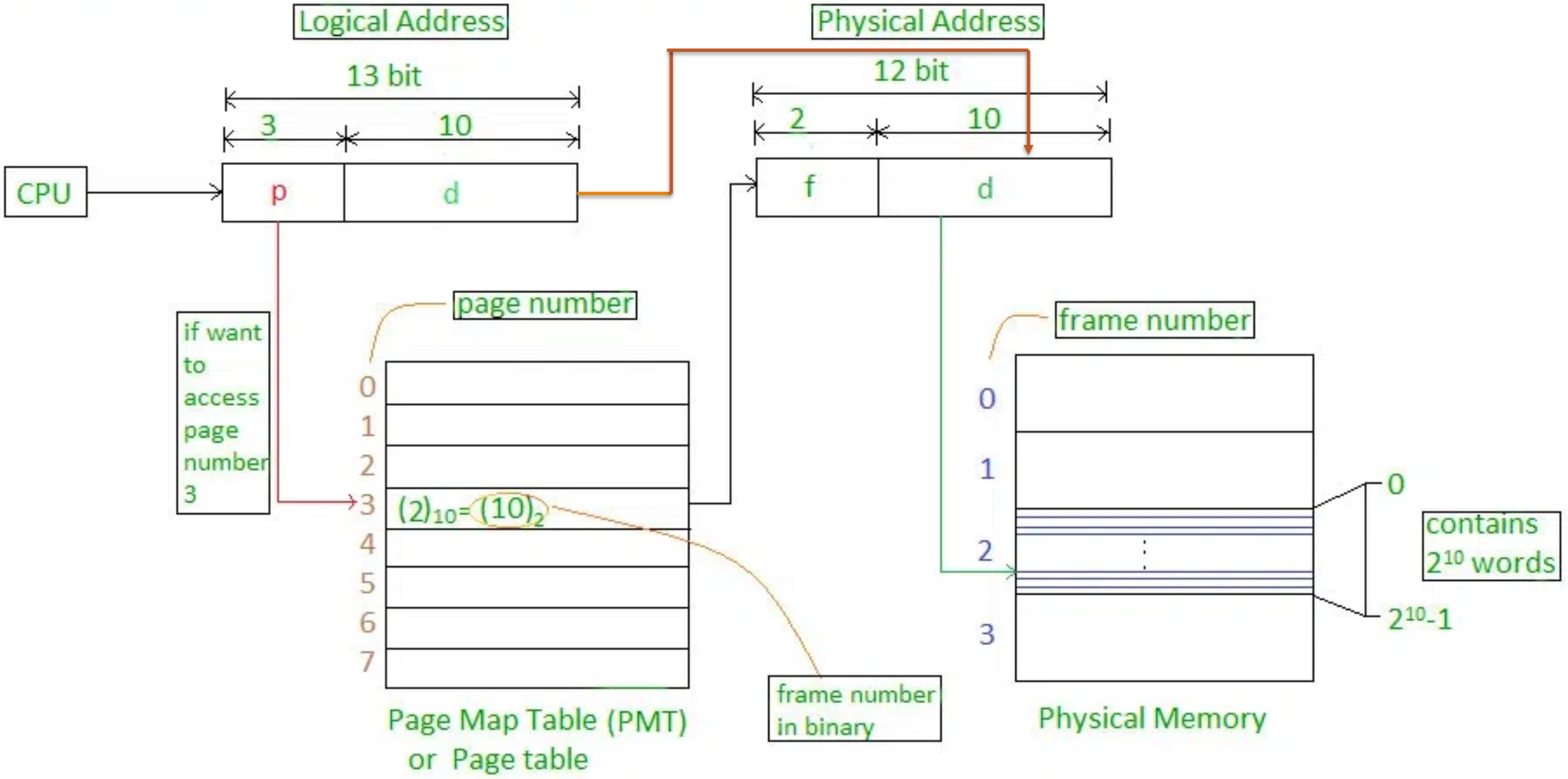
- Paging is a memory management scheme that eliminates the need for a contiguous allocation of physical memory.
- Paging is a storage mechanism used in OS to retrieving processes from secondary storage to main memory as pages is known as paging.
- The basic purpose of paging is to separate each procedure into pages. Additionally, frames will be used to split the main memory. This scheme permits the physical address space of a process to be non – contiguous.

Let us consider an example:
•Physical Address = 12 bits, then Physical Address Space = 4 K words
•Logical Address = 13 bits, then Logical Address Space = 8 K words
•Page size = frame size = 1 K words (assumption)

Paging Concept:

Number of frames = Physical Address Space / Frame size = 4 K / 1 K = 4 = 2^2

Number of pages = Logical Address Space / Page size = 8 K / 1 K = 8 = 2^3



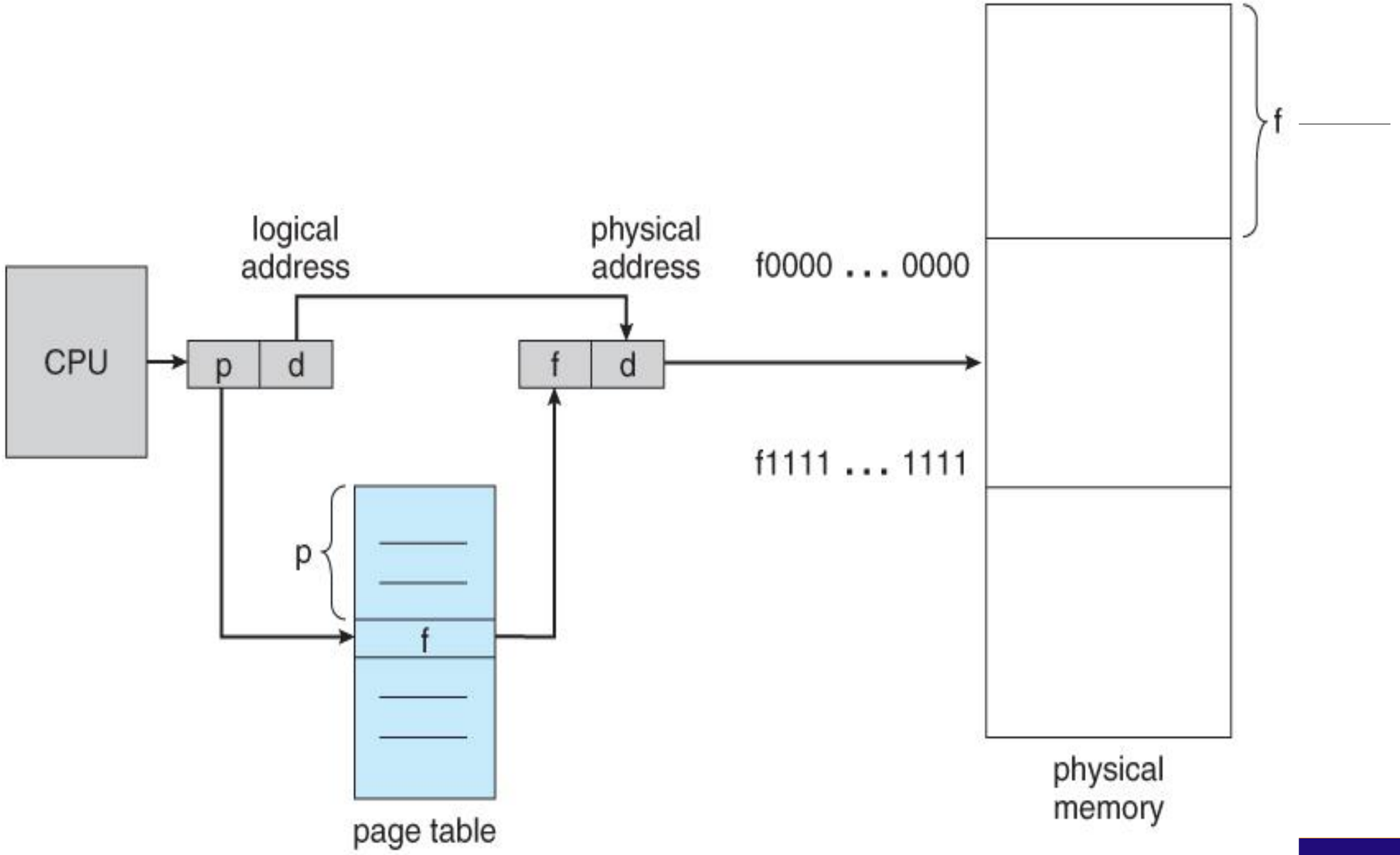
Paging: Concept, Structure of Page table

- In paging, the physical memory is divided into fixed-size blocks called frames, which are the same size as the pages used by the process.
- The process's logical address space is also divided into fixed-size blocks called pages, which are the same size as the page frames.
- When a process requests memory, the operating system allocates one or more page frames to the process and maps the process's logical pages to the physical page frames.

Paging

- Allows the physical address space to be non contiguous.
- Eliminates problems with fragmentation.
- Divide physical memory into a number of equal sized blocks called ***frames***
- Divide a programs logical memory space into blocks of the same size called ***pages.***
- ***Page table*** is used to look up what frame a particular page is stored in at the moment

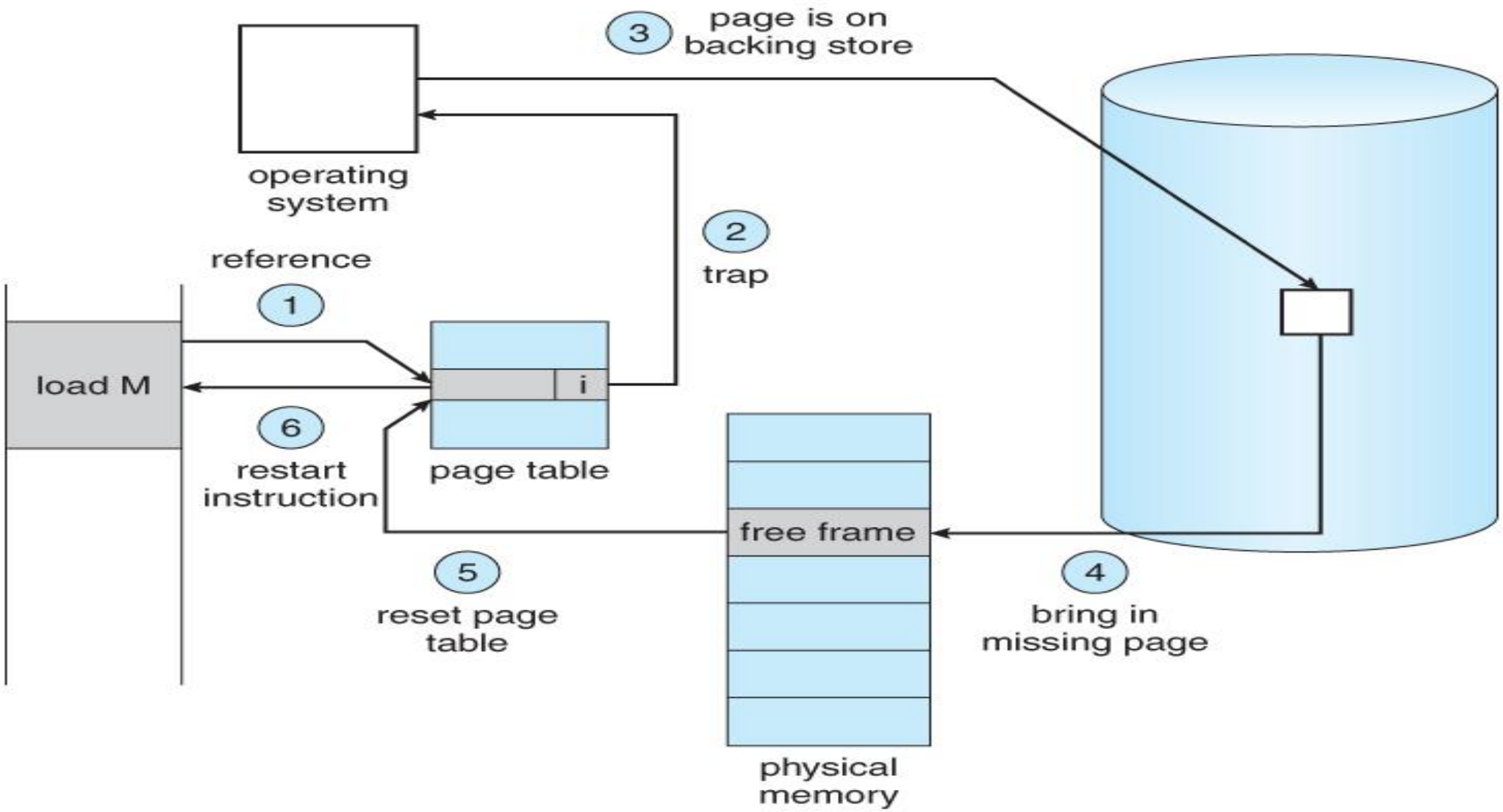
Paging hardware: Diagram



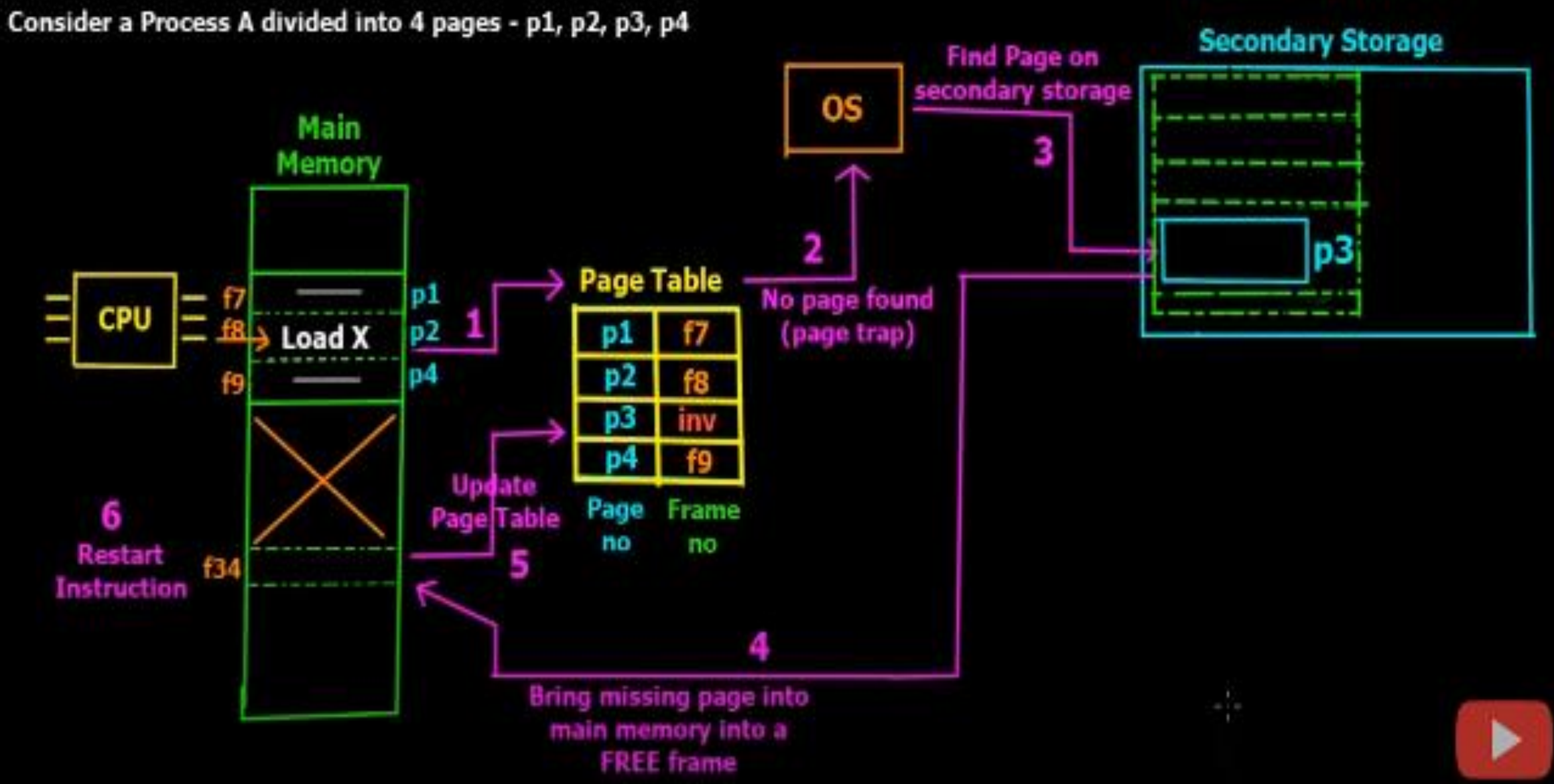
Page fault

- Page fault is a type of error that occurs when a program tries to access data that is not currently in the main memory or random access memory (RAM).
- A page fault occurs when a program attempts to access data or code that is in its address space, but is not currently located in the system RAM. But what happens if the process tries to access a page that was not brought into memory? Access to a page marked invalid causes a Page fault.
- The paging hardware, in translating the address through the page table, will notice that the invalid bit is set, causing a trap to the operating system.

Page fault handling



Page Faults & Page Fault Handling | Thrashing



Page fault handling

The procedure for handling this page fault is straightforward (following Figure):

1. We check an internal table (usually kept with the process control block) for this process to determine whether the reference was a valid or an invalid memory access.
2. If the reference was invalid, we terminate the process. If it was valid, but we have not yet brought in that page, we now page it in.
3. We find a free frame (by taking one from the free-frame list, for example).
4. We schedule a disk operation to read the desired page into the newly allocated frame.
5. When the disk read is complete, we modify the internal table kept with the process and the page table to indicate that the page is now in memory.
6. We restart the instruction that was interrupted by the trap. The process can now access the page as though it had always been in memory.

Segmentation

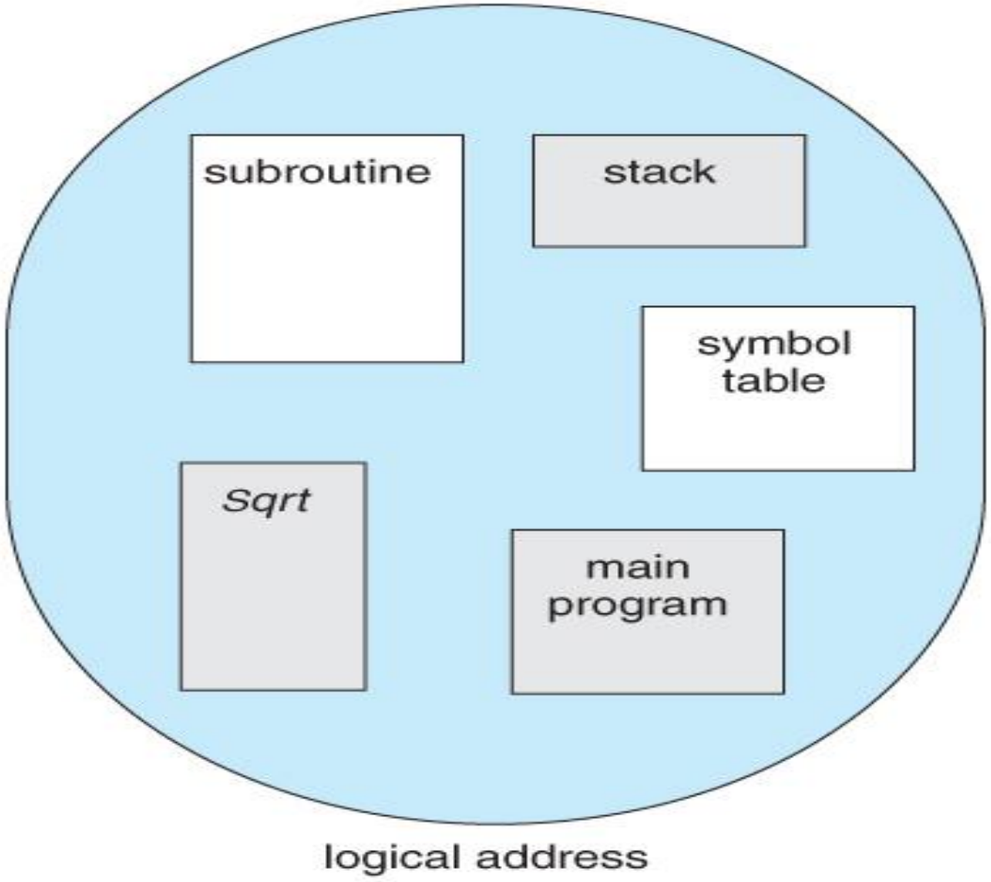
- In Operating Systems, Segmentation is a memory management technique in which the memory is divided into the variable size parts. Each part is known as a segment which can be allocated to a process.

What is Segment Table?

It maps a two-dimensional Logical address into a one-dimensional Physical address. It's each table entry has:

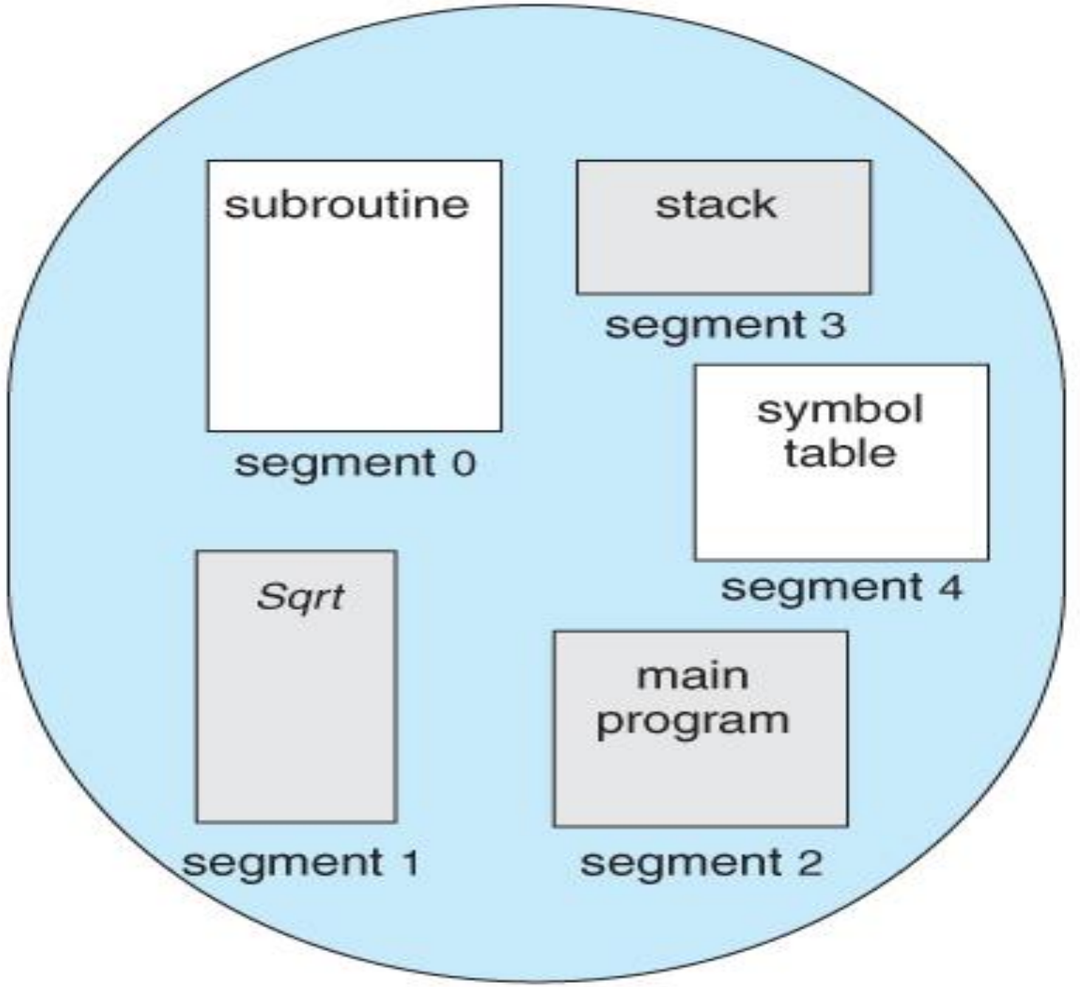
- **Base Address:** It contains the starting physical address where the segments reside in memory.
- **Segment Limit:** Also known as segment offset. It specifies the length of the segment.

Segmentation



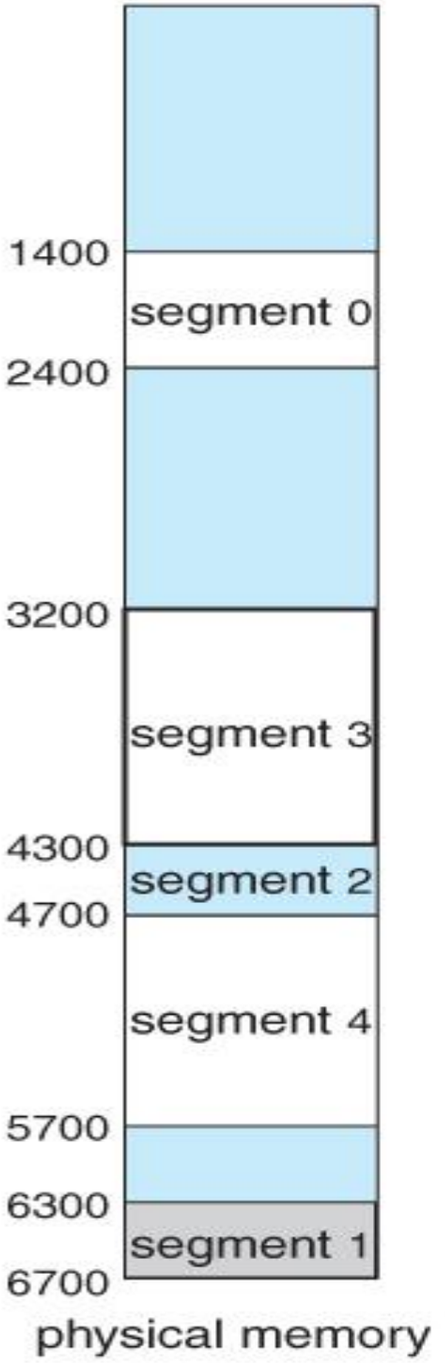
Programmer's view of a program.

Example of segmentation



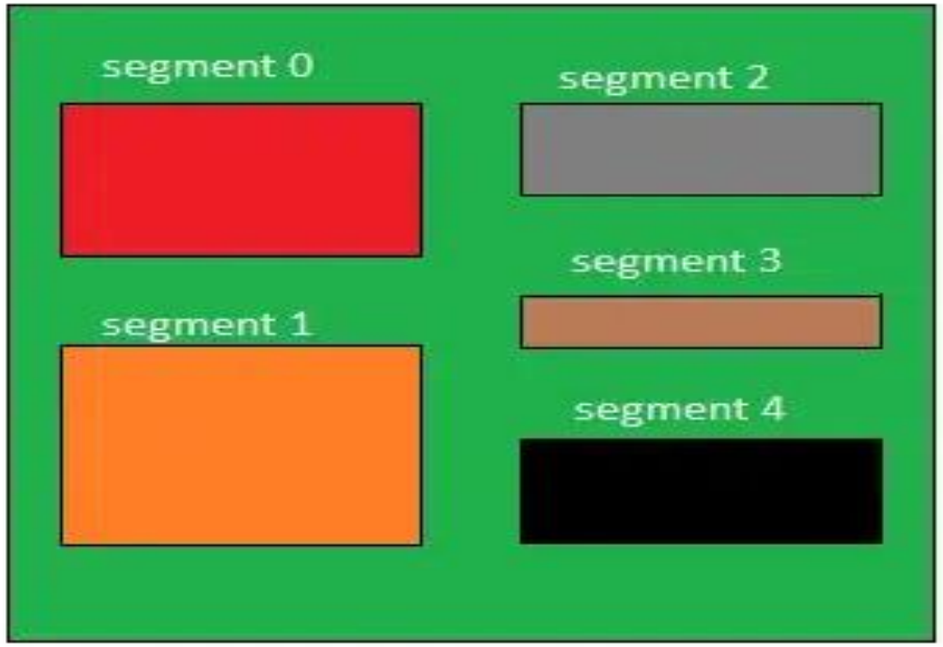
	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

segment table



#Educatio

Logical View of Segmentation

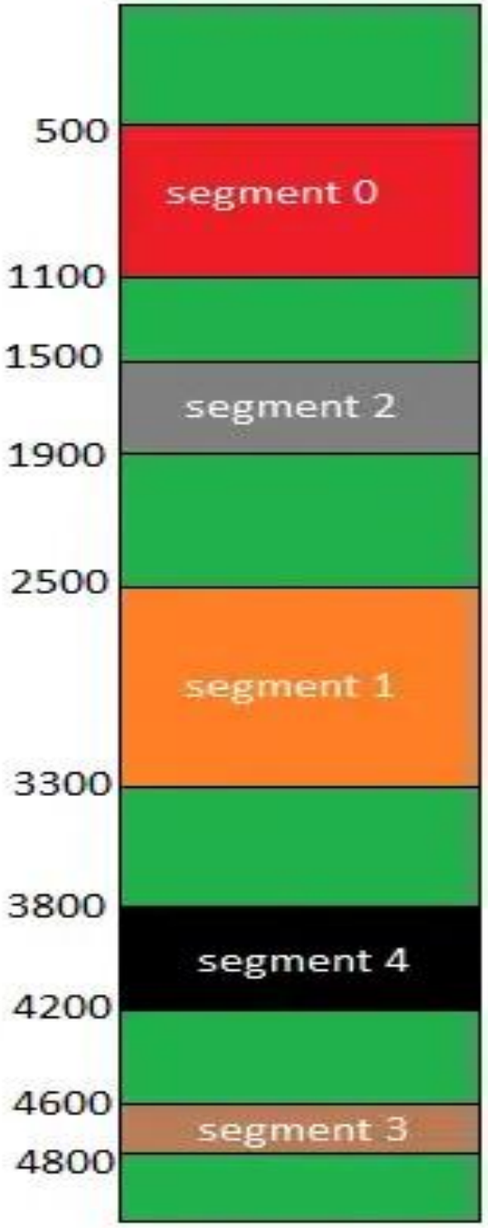


Logical Address Space

Segment Number

	base address	Limit
0	500	600
1	2500	800
2	1500	400
3	4600	200
4	3800	400

Segment Table



Physical Address Space

Paging vs. Segmentation

Paging

- Invisible from the user program
- Divides physical memory into blocks (pages)
- Pages are of uniform, predetermined size
- Logical address is a positive integer

Segmentation

- Visible to the user program
- Divides logical memory into blocks (segments)
- Segments are of arbitrary sizes
- Logical address is an ordered pair: (segment, offset)

In both cases, the operating system works together with the hardware to translate logical addresses into physical addresses.

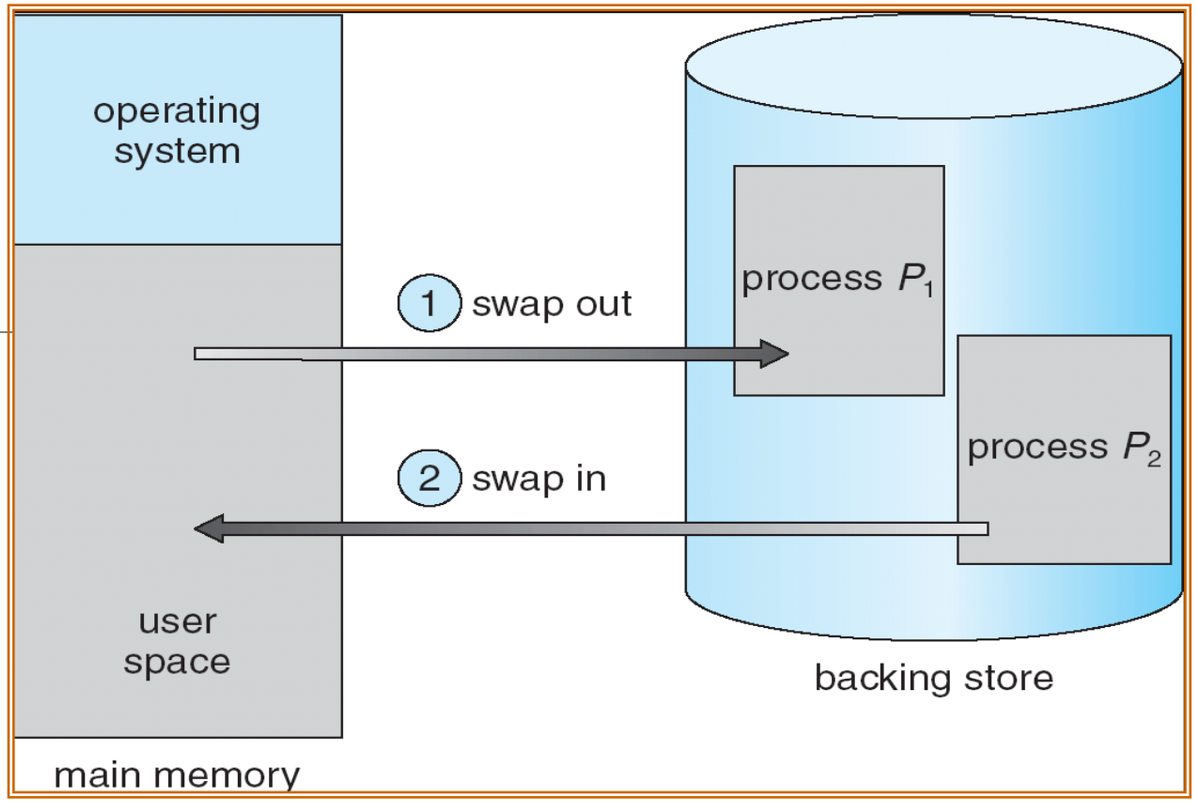
Swapping

Swapping in an operating system involves moving data between RAM and disk to manage memory efficiently,

- A process must be in memory to be executed.
- A process can be **swapped temporarily out of memory to a backing store,** and then brought back into memory for continued execution.
- Total physical memory space of processes can exceed physical memory
- For example, assume a multiprogramming environment with a round-robin CPU scheduling algorithm. When a quantum expires, the memory manager will start to swap out the process that just finished and to swap another process into the memory pace that has been freed.

Swapping

- **Backing store** is fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- Roll out, roll in – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed



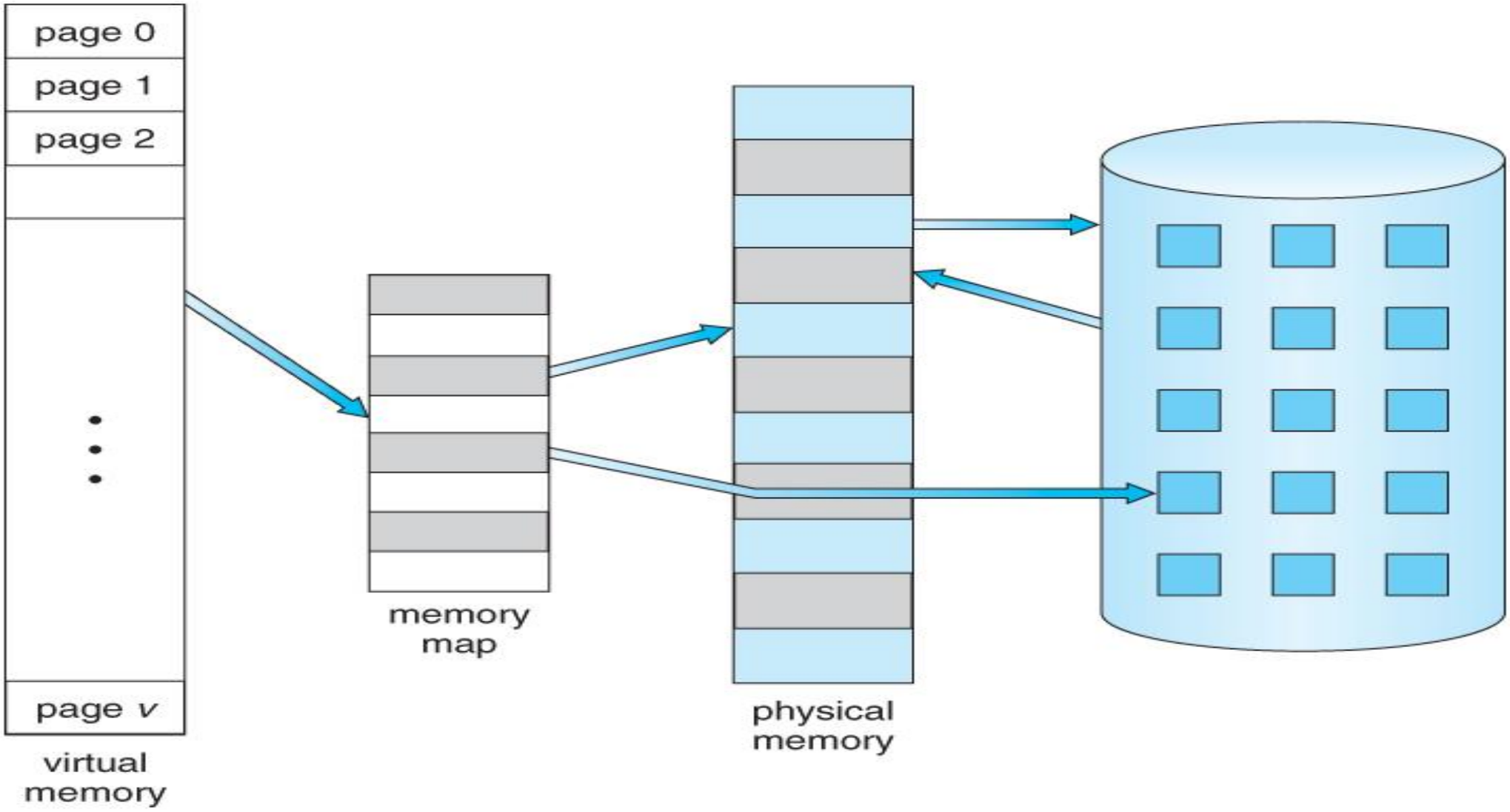
Virtual Memory

- Virtual Memory is a technique that allows the execution of processes that **may not completely in memory**.
- Virtual Memory is a storage scheme that provides **user an illusion** of having a very big main memory. This is done by treating a part of **secondary memory** as the main memory.
- It provides the illusion to the programmer that the process who's size is larger than the main memory can also be execute.

Benefits:

- Programs could be written for a much larger address space (virtual memory space) than physically exists on the computer.
- Improve CPU utilization and system throughput.
- Less I/O is needed for swapping processes in and out of RAM, speeding things up.

Virtual Memory



Page replacement:Overview

- In an operating system, page replacement refers to a scenario in which a page from the main memory should be replaced by a page from the secondary memory.
- Page replacement occurs due to page faults.
- **Over-allocation of memory itself causes problem as : While a user process is executing, a page fault occurs. The operating system determines where the desired page is residing on the disk but then finds that there are no free frames on the free-frame list; all memory is in use.**

Page replacement:Overview

Then the question arises as :

What Happens if There is no Free Frame and still another page is to be brought into memory?

The solution to this problem : OS could swap out unused pages on disk and swap in required pages in main memory.

Again here question is : Which page to replace (swap out)

Solution is : it is decided by the Page Replacement Algorithm.

Basic Page Replacement:

1. Find the location of the desired page on the disk.
2. Find a free frame:
   - a. If there is a free frame, use it.
   - b. If there is no free frame, use a page-replacement algorithm to select
   - c. Write the victim frame to the disk; change the page and frame tables

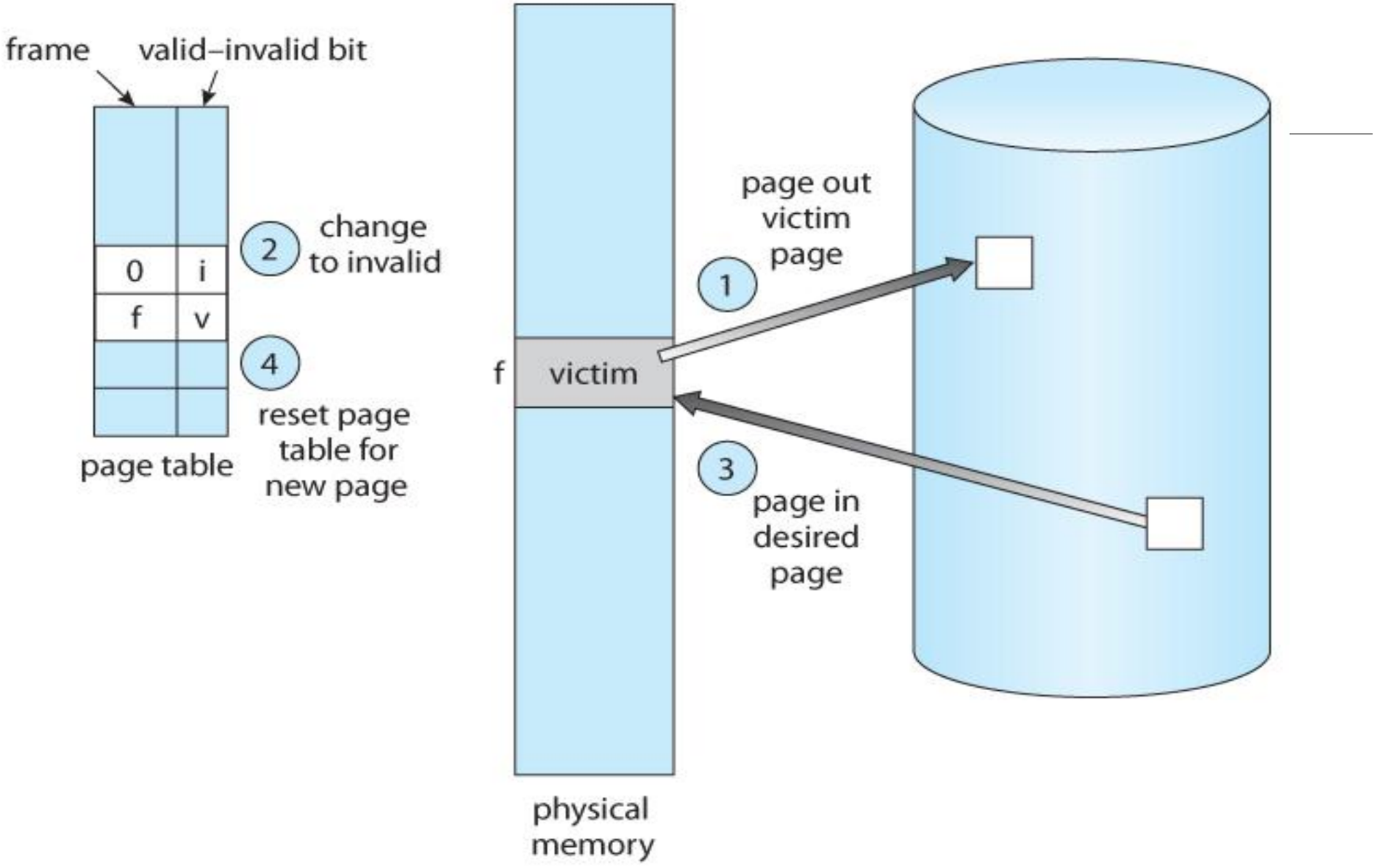
accordingly.

3. Read the desired page into the newly freed frame; change the page and

frame tables.

4. Restart the user process.

Page replacement



Page replacement algorithms

- FIFO (First in First out)
- Optimal
- LRU (Least Recently Used)

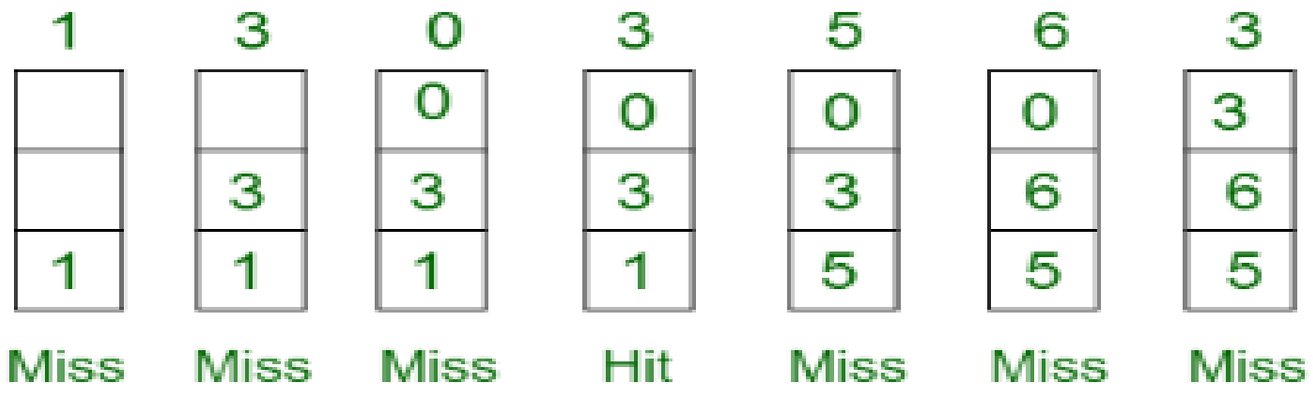
FIFO Page Replacement:

- The simplest page-replacement algorithm is a **first-in, first-out (FIFO)** algorithm.
- A FIFO replacement algorithm associates with each page the time when that **page was brought into memory**.
- When a page must be **replaced**, the **oldest page** is chosen. We can create a FIFO queue to hold all pages in memory.
- We replace the page at the head of the queue. When a page is brought into memory, we insert it at the tail of the queue.

FIFO Page Replacement- Example1

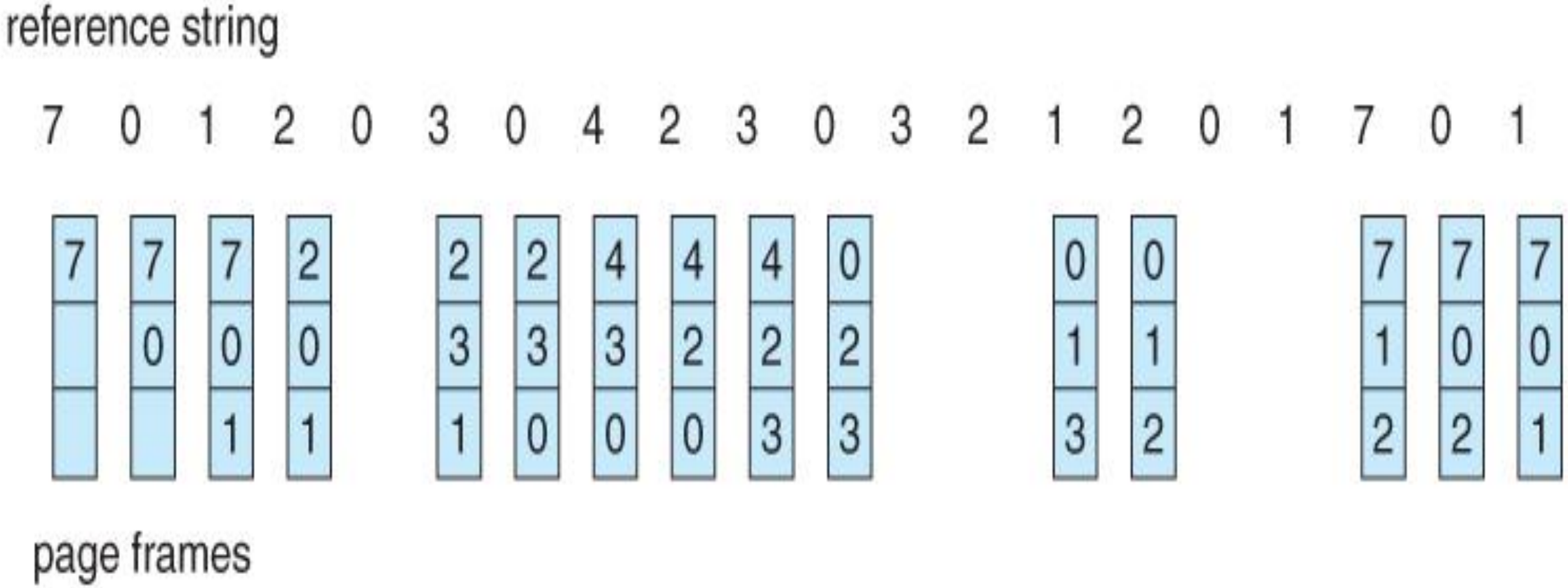
Example 1: Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3 page frames. Find the number of page faults.

Page reference 1, 3, 0, 3, 5, 6, 3



Total Page Fault = 6

FIFO Page Replacement



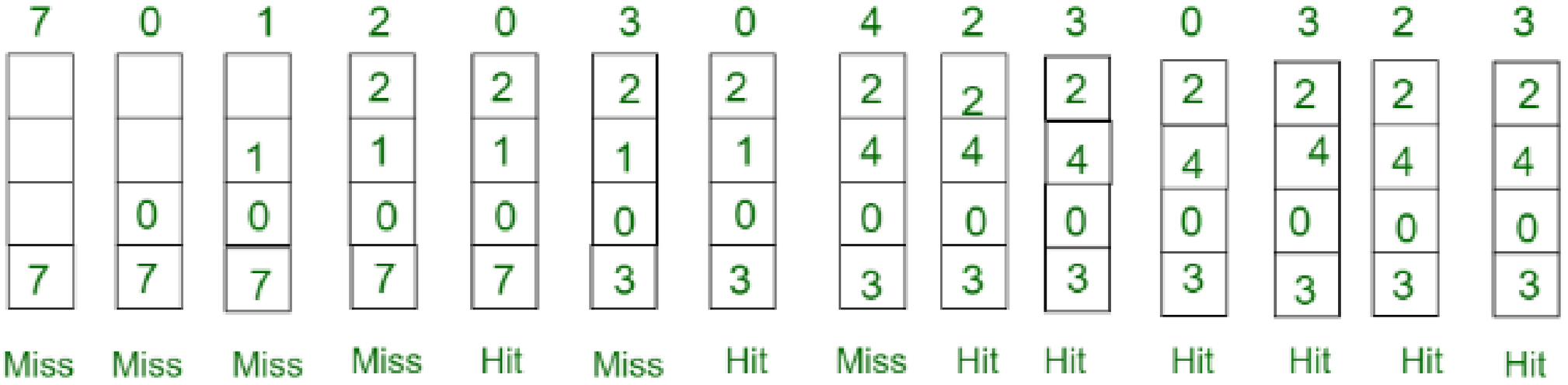
OPT (Optimal) Page Replacement

- The key distinction between the FIFO and OPT algorithms (other than looking backward versus forward in time) is that the **FIFO algorithm uses the time** when a page was brought into memory, **whereas the OPT algorithm uses the time when a page is to be used.**
- Again here, it is assumed that main memory has three frames to solve following sum.

Optimal Page Replacement- Example

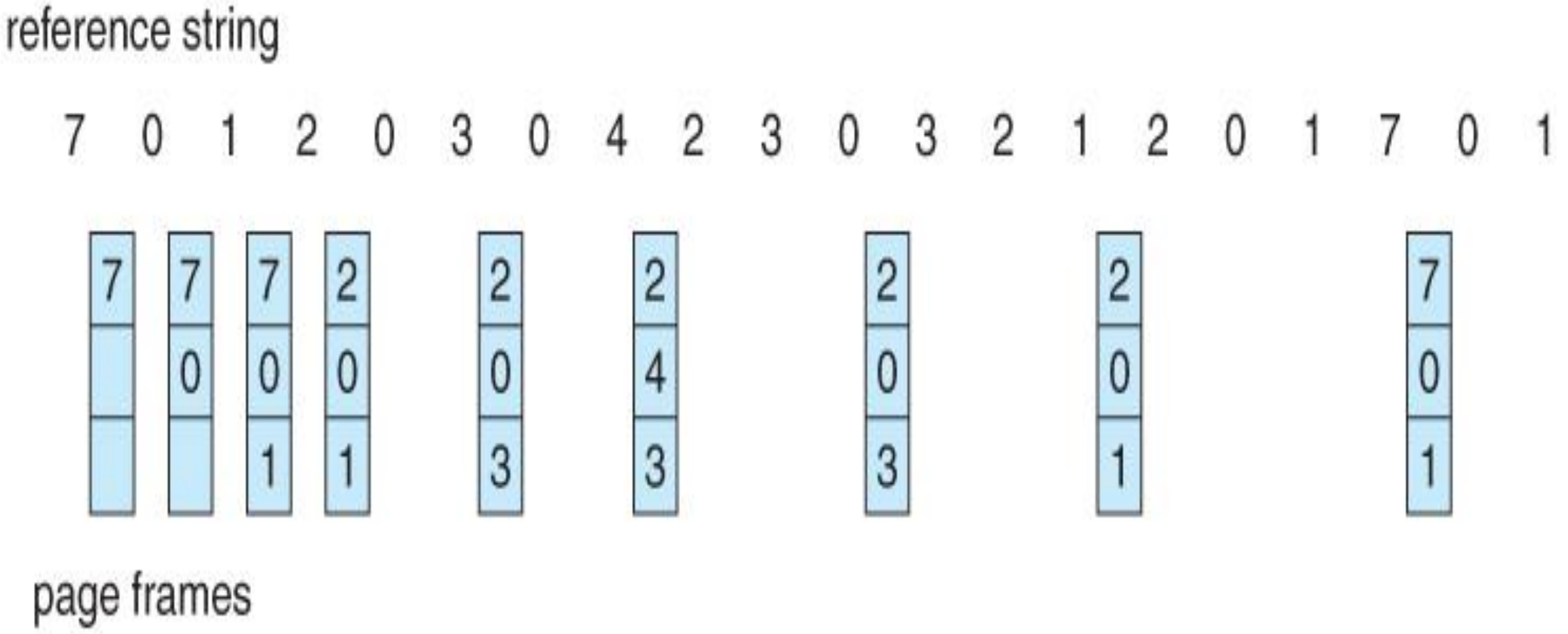
Example-2: Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4 page frame. Find number of page fault.

Page reference 7,0,1,2,0,3,0,4,2,3,0,3,2,3 No. of Page frame - 4



Total Page Fault = 6

Optimal Page Replacement- Example



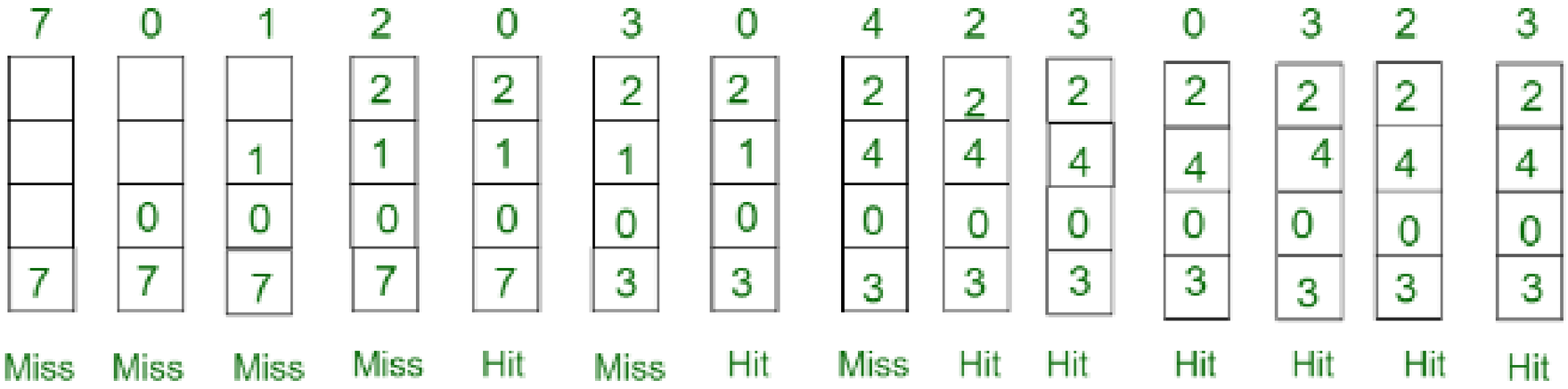
LRU Page Replacement

- **LRU is** : Replace the page that has not been used for the **longest period** of time.
- Use **past knowledge** rather than **future**. Associate time of last use with each page.
- Again here, it is assumed that main memory has three frames to solve following sum.

LRU Page Replacement- Example

Example-3: Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4 page frames. Find number of page faults.

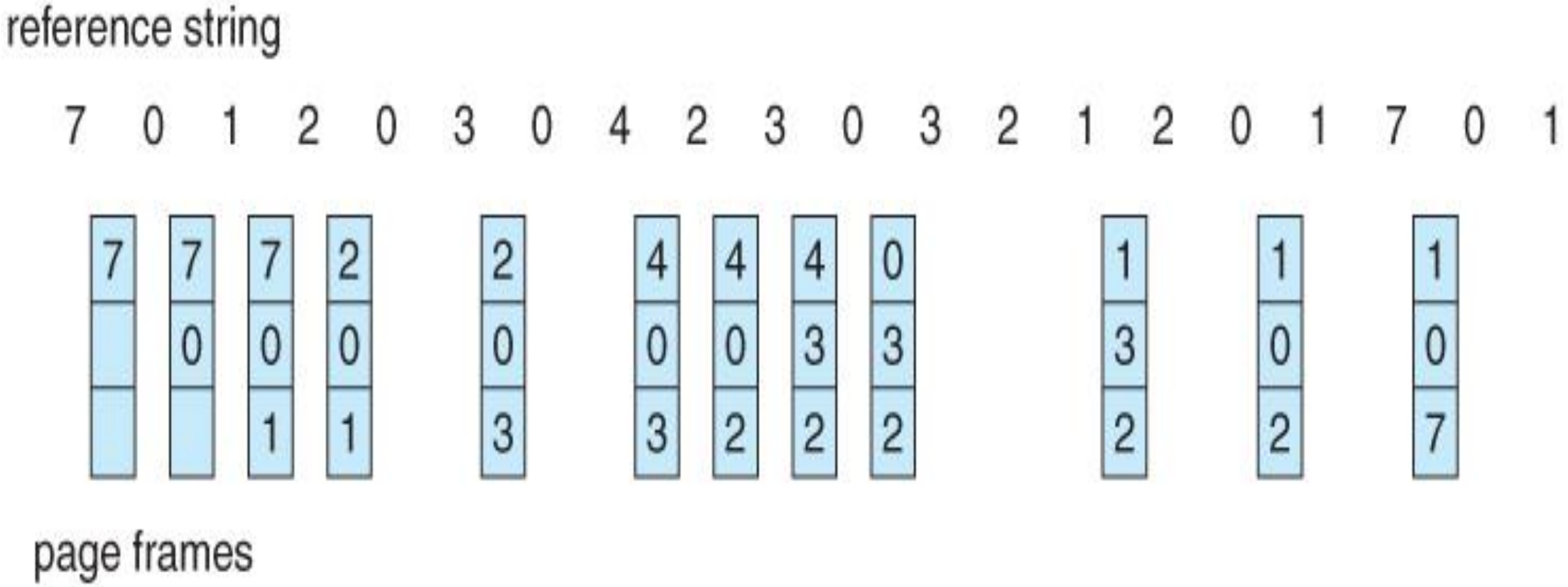
Page reference 7,0,1,2,0,3,0,4,2,3,0,3,2,3 No. of Page frame - 4



Total Page Fault = 6

Here LRU has same number of page fault as optimal but it may differ according to question.

LRU Page Replacement- Example



THANK YOU